

CSC 330 Programming Languages

Lecture 8: Logic Paradigm - Prolog

Introduction

Prolog is a logic based language. In fact, the name stands for PROgramming in LOGic. Prolog can be thought of as a first generation *attempt* to create a declarative language. The idea behind declarative languages is that we give the computer *definitions* – descriptions of **what** the problem is, rather than *algorithms* – instructions detailing **how** to solve a problem. This goal is only partially attained in Prolog, to write useful programs it's necessary to understand the procedural nature of the Prolog interpreter.

A Prolog program is a collection of clauses. There are three types of clauses: **facts**, **rules**, and **queries** (also known as questions). In general, facts and rules are used to define relations (such as parent, sibling, child, etc.) while queries are used to ask questions to Prolog about the relations. But before we dive any deeper into that, let's get Prolog installed.

Installation

To run Prolog, you will need a Prolog interpreter. SWI Prolog is a very good choice and is what I will be using in class. It's available at <https://www.swi-prolog.org/download/stable>. There are versions for both Windows and Mac. After installing, go ahead and open up the software.

Create a file called "main.pl" (or really whatever you want to call it as long as it has the ".pl" extension). This is where your "facts" and "rules" are going to go, essentially creating your database of knowledge. SWI Prolog is used as a compiler for these facts and rules, but also as a query terminal to run queries/questions at the "?-" prompt. It also has a built in text editor that you are free to use, but I'll be using Notepad++ in class to edit the file.

To load our "main.pl" file we first need to navigate to the directory where the file is stored. We can use "cd" command for this, but this command is slightly different than you are used to. Type the following into the SWI prompt:

```
cd('fullpath/to/your/folder/with/prolog/file/').
```

with the path in quotes replaced with the path to the directory containing "main.pl". Make sure to not put a space between "cd" and the opening paren. Notice the dot at the end, that's important. It essentially serves the same purpose as semi colon in Java. After this, you can load your file anytime with the following:

```
[main].
```

Or you can use:

```
consult(main).
```

This is given that you named your file "main.pl", so it is basically the file name without the file extension. Your file should be empty at this point, so nothing interesting happens. Every time you make a change to your file, you will need to reload it. Just like with other terminals, you are allowed to use up/down arrow keys to cycle through your command history.

Facts

A fact represents a statement that is always unconditionally true. For example:

```
teacher(kevin) .
```

This fact means that kevin is a teacher. In this clause, "kevin" is said to be an *atom*. An atom is essentially a value. This concept is similar to that of symbols from Scheme. Atoms are NOT variables and can not be used to hold data, instead, they essentially ARE the data. In the clause above, kevin is now introduced as a valid value and it is a true to say that this value, kevin, is a teacher. We could have also written this as:

```
teacher('kevin') .
```

In Prolog, surrounding something in single quotes is the same as not putting the quotes at all, both would be considered atoms. In fact, we can verify this in SWI by typing the following at the prompt:

```
?- 'kevin' = kevin.
```

We get a result of "true" from this. Note, however, that using double quotes creates a string, which is NOT the same as an atom:

```
?- "kevin" = kevin.
```

This gives us "false" as the result.

Facts can show relationships among atoms as well. For example:

```
parent(tom, bob) .
```

This fact defines a relation named "parent". We would read this as: "Tom is a parent of Bob." The fact has two *arguments*, the first is the atom 'tom' and the second is the atom 'bob'. Notice that both of these arguments begin with a lower case letter, just like kevin did. This is important since variables, as we will see shortly, always begin with an upper case letter.

All facts end with a period (so do queries and rules as we will see later). A collection of facts (and rules) form a "database". The following is a database that consists of six facts.

```
parent(pam, bob) .      % Pam is the parent of Bob.  
parent(tom, bob) .  
parent(tom, liz) .  
parent(bob, ann) .  
parent(bob, pat) .  
parent(pat, jim) .
```

This describes a part of a family tree. Prolog databases are *somewhat insensitive* to the order that the clauses appear – especially when the database consists only of facts with constant arguments (aka atoms). The database given above would "mean" the same thing even if the order of the facts were changed. Note that this makes sense for definitions (if you tell someone two separate facts, normally the order does not matter) but not for algorithms (where the order in which you do things is typically *very* important).

Queries

Once a database has been defined, we may then ask questions about those relationships. These questions are typically called *queries*. After writing your facts into a file and loading that file in SWI, you can type queries into the SWI prompt. Here are some examples:

To ask "Is Bob a parent of Pat?" we would type:

```
?- parent(bob,pat) .
```

To which SWI Prolog would respond with:

```
?- parent(bob,pat) .  
true.
```

Some systems would put "yes" instead of "true". To ask "Is Liz a parent of Pat?" we would type:

```
?- parent(liz,pat) .
```

To which Prolog would respond with:

```
?- parent(liz,pat) .  
false.
```

To ask "Is Tom a parent of Ben?" we would type:

```
?- parent(tom,ben) .  
false.
```

The reason Prolog responded "false" to the last query was that it had never heard of ben. Prolog assumes that it has been given *all* of the relevant information when we load the database. Since it had not been told about a person named ben who was a child of tom, it assumed that no such child exists. This assumption that the system has all relevant knowledge is called the "closed world assumption".

We often want to form queries that require more than a "true" or "false" answer. Fortunately, Prolog has an extremely powerful algorithm for deduction. We not only can ask if something is true, we can also ask what would make something true. In other words, what values would make this query true. For example, to ask "who is/are the parent(s) of Liz?", we would type:

```
?- parent(X,liz) .
```

Prolog would respond with:

```
X = tom.
```

Since X is capitalized, Prolog treats this as a *variable*. Any text would work here as long as the initial letter is capitalized. You can interpret the answer given by Prolog to mean that when X is given the value of 'tom', then the query becomes true, since tom is a parent of liz. Prolog is essentially finding any and all values for X that would make the query true.

Frequently, a query will have more than a single answer. For example, to ask "Bob is the parent of whom?", we would type:

```
?- parent(bob, X) .
```

To which Prolog would respond with:

```
X = ann
```

Now, instead of hitting enter, type a semicolon. Prolog now responds with

```
X = pat.
```

The period at the end means that Prolog knows this is the last answer. Entering a semicolon after Prolog has given you an answer is your way of asking Prolog to see if it can find another answer to the same query. If it can, it will give you that answer. You can continue to request additional answers and as long as Prolog can find them it will give them to you. Eventually, Prolog will either know there are no more answers causing it to generate a period "." or it will have exhausted all possibilities and will respond **false**. meaning there are no more answers that Prolog can find.

Is it possible to ask Prolog to find ann's grandparents? Yes, it is. The approach that we must take is to first find a parent of ann and then find a parent of that parent. We can express this idea in Prolog as a query that contains two parts or "goals". Separating them with a comma means that any potential solution must satisfy both goals.

To ask, "Who is/are Ann's grandparents?", we type:

```
?- parent(X, ann), parent(Y, X) .
```

Prolog would respond with:

```
X = bob, Y = pam
```

If you then enter a semicolon ; Prolog would respond with:

```
X = bob, Y = tom.
```

Spacing and Duplicates

Prolog doesn't care about spacing for the most part. This means that the following are all equivalent:

```
parent( bob, pat) .  
parent( bob,   pat) .  
parent( bob    ,pat) .  
parent(bob,    pat  ) .  
parent(   bob      ,    pat    )      .
```

The one place it does care about spacing is right before the opening paren. The following won't compile:

```
parent (bob, pat) .
```

Upon compilation, we get:

Syntax error: Operator expected

It's also interesting to note that Prolog considers facts to be separate, even if they are the same fact repeated. For example, if we have the following in our database:

```
parent(bob, pat).
```

And then ask who the parent of pat is, we get bob with a period after it, indicating that there are no more answers:

```
?- parent(X, pat).  
X = bob.
```

However, if we duplicate the fact in our database and reload it:

```
parent(bob, pat).  
parent(bob, pat).
```

Then we ask who the parent of pat is, we get:

```
?- parent(X, pat).  
X = bob ;  
X = bob.
```

Entering semi colon reveals the second X = bob answer. So, Prolog doesn't naturally remove duplicate entries in our database.

Rules

A fact is a statement that is always unconditionally true. A *rule* on the other hand, is a statement that is conditionally true. In other words, a rule defines a relation that will be true if some condition is satisfied. Normally this is written as:

```
IF condition THEN conclusion
```

In Prolog the form of a rule is:

```
conclusion :- condition.
```

The conclusion or HEAD of the rule will be true if the condition or BODY of the rule is true. For example, to express the rule:

```
IF X is a parent of Y THEN Y is a child of X
```

We would write:

```
childof(Y, X) :- parent(X, Y).
```

Which we read as, "Y is a child of X, if X is a parent of Y."

General Behavior of Prolog Programs

As we have seen, Prolog programs consist of facts and rules. We "run" a program by loading the database and then posing one or more queries to Prolog. Prolog attempts to answer our queries by examining the facts and rules it has been given. The following example will illustrate the general procedure Prolog uses.

Assume Prolog has been loaded with these facts and rules:

```
parent(pam, bob).
parent(tom, bob).
parent(tom, liz).
parent(bob, ann).
parent(bob, pat).
parent(pat, jim).

childof(Y, X) :- parent(X, Y).
```

And we pose the following query to it:

```
?- childof(liz, tom).
```

Here is how Prolog answers our query:

1. First, Prolog establishes the query as the *current goal*.
2. Next, Prolog looks for any FACTs that would satisfy the current goal, examining them in the order they appear. In this case there are no facts that pertain to "childof".
3. Prolog then looks at RULEs that conclude things about the current goal, examining them in the order they appear. In this case there is only one rule that pertains to "childof".
4. Prolog then selects this rule (if there had been more than a single rule, Prolog would have grabbed the first one) and binds the variables (which is a process called "unification"). In other words, Y is unified to 'liz' and X is unified to 'tom', i.e.

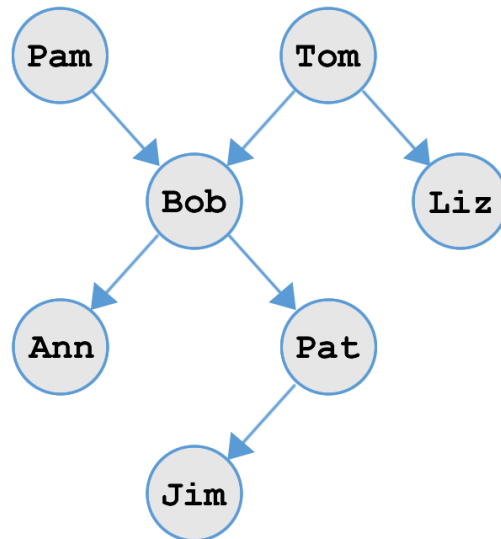
```
childof( Y,    X )
      =      =
childof(liz, tom)
```

5. The condition part (body) of the rule now becomes the new goal (or set of goals if the rule had contained multiple conditions) and we begin again at Step 2 of this process, i.e., look for any fact(s) that satisfies the current goal: "parent(tom,liz)". In this case, we do in fact find parent(tom,liz) as a fact, and so Prolog concludes that this unification results in a true statement.

Here is a more complete Prolog database showing many more family relationships:

```
% Family Relationships (the concept that children have two parents is not modeled)
```

```
parent(pam,bob).  
parent(tom,bob).  
parent(tom,liz).  
parent(bob,ann).  
parent(bob,pat).  
parent(pat,jim).
```



```
female(pam).  
female(liz).  
female(ann).  
female(pat).
```

```
male(tom).  
male(bob).  
male(jim).
```

```
offspring(X,Y) :- parent(Y,X).  
childof(X,Y) :- offspring(X,Y).
```

```
mother(X,Y) :- parent(X,Y), female(X).  
father(X,Y) :- parent(X,Y), male(X).
```

```
grandparent(X,Z) :- parent(X,Y), parent(Y,Z).  
grandmother(X,Z) :- grandparent(X,Z), female(X).  
grandfather(X,Z) :- grandparent(X,Z), male(X).  
grandchild(X,Y) :- grandparent(Y,X).
```

```
sibling(X,Y) :- parent(Z,X), parent(Z,Y), X\=Y.
```

```
sister(X,Y) :- parent(Z,X), parent(Z,Y), female(X), X\=Y.  
sister2(X,Y) :- sibling(X,Y), female(X).
```

```
brother(X,Y) :- parent(Z,X), parent(Z,Y), male(X), X\=Y.  
brother2(X,Y) :- sibling(X,Y), male(X).
```

```
aunt(X,Y) :- parent(Z,X), parent(Z,P), parent(P,Y), female(X), X\=P.  
aunt2(X,Y) :- sister(X,P), parent(P,Y).
```

```
uncle(X,Y) :- parent(Z,X), parent(Z,P), parent(P,Y), male(X), X\=P.  
uncle2(X,Y) :- brother(X,P), parent(P,Y).
```

```
cousin(X,Y) :- parent(A,X), parent(B,Y), sibling(A,B).
```

```
% The following shows you how to climb a hierarchy.
```

```
ancestor(X,Z) :- parent(X,Z).  
ancestor(X,Z) :- parent(X,Y), ancestor(Y,Z).
```

A (Somewhat) Deeper Understanding of Prolog

While Prolog "aims" to be a declarative language, it really falls far short of that goal¹. The order of facts, rules, and goals within a query is, in reality, critically important to writing a successful program.

In order to write Prolog programs that actually compute answers (rather than answer questions over a knowledge base) it is best to think of these programs as RECURSIVE procedures or functions.

```
% A procedurally correct and efficient version of ancestor.
ancestor(X,Z) :- parent(X,Z).                % Rule (1)
ancestor(X,Z) :- parent(X,Y), ancestor(Y,Z). % Rule (2)

% Ask PROLOG whether tom is an ancestor of pat
?- ancestor(tom,pat).

% Try Rule (1) where we unify X with 'tom' and Z with 'pat'
TRY Rule(1)      ancestor(tom,pat) :- parent(tom,pat).

% To conclude predecessor(tom,pat) we need to show parent(tom,pat) is true
TRY              parent(tom,pat).

% There is no parent(tom,pat) fact, nor any rules that conclude "parent"
FAIL              parent(tom,pat).

% Since Rule(1) was a bust, we try to prove ancestor(tom,pat) via Rule(2)
TRY Rule(2)      ancestor(tom,pat) :- parent(tom,Y), ancestor(Y,pat).

% Rule(2) requires we show TWO things true. The first being parent(tom,Y)
TRY              parent(tom,Y).

% We found a fact showing that tom is bob's parent, so Y is unified to 'bob'
FOUND FACT       parent(tom,bob).

% To conclude ancestor(tom,pat), we must show the second clause is true
TRY              ancestor(bob,pat).

% We now go back to rule(1); note that this is really a recursive call.
TRY Rule(1)      ancestor(bob,pat) :- parent(bob,pat).

% To conclude ancestor(bob,pat) we need to show parent(bob,pat) is true
TRY              parent(bob,pat).

% We found a fact showing that bob is pat's parent.
FOUND FACT       parent(bob,pat).

% Thus, we can conclude that Rule(1) where X is 'bob' and Z is 'pat' is true.
PROVED RULE(1)   ancestor(bob,pat) :- parent(bob,pat).

% We pop up a level of recursion back to Rule(2) which we now know to be true
PROVED RULE(2)   ancestor(tom,pat) :- parent(tom,bob), ancestor(bob,pat).
```

¹ This is a problem that is actively being worked on: <https://youtu.be/Ru2oZGRJTQg?si=EUL3QeLdYgi88i0p>

A good general guideline for writing procedurally efficient and correct programs is to try the simplest cases first. Organize your Prolog code in the following way:

1. Facts first.
2. Rules that are non-recursive.
3. Rules that are recursive. Place the recursive clause as far to the right as possible (Prolog doesn't like left recursive rules too much).

Input is Output and Output is Input

A unique feature of Prolog is that, unlike most every other programming language, we do not specify which variables are input and which are output. This means that programs can often "run in reverse" of the way we typically think of them running.

```
% Concatenates two lists together and returns the result in a third list.
```

```
concatenate([], L, L).
```

```
concatenate([X|L1], L2, [X|L3]) :- concatenate(L1, L2, L3).
```

```
[trace] ?- concatenate([a,b,c],[x,y,z],Output).
Call: (1) concatenate([a, b, c], [x, y, z], _6186)
Call: (2) concatenate([b, c], [x, y, z], _6434)
Call: (3) concatenate([c], [x, y, z], _6440)
Call: (4) concatenate([], [x, y, z], _6446)
Exit: (4) concatenate([], [x, y, z], [x, y, z])
Exit: (3) concatenate([c], [x, y, z], [c, x, y, z])
Exit: (2) concatenate([b, c], [x, y, z], [b, c, x, y, z])
Exit: (1) concatenate([a, b, c], [x, y, z], [a, b, c, x, y, z])
Output = [a, b, c, x, y, z].
```

```
[trace] ?- concatenate(Output, [x,y,z], [a,b,c,x,y,z]).
Call: (1) concatenate(_6200, [x, y, z], [a, b, c, x, y, z])
Call: (2) concatenate(_6480, [x, y, z], [b, c, x, y, z])
Call: (3) concatenate(_6486, [x, y, z], [c, x, y, z])
Call: (4) concatenate(_6492, [x, y, z], [x, y, z])
Exit: (4) concatenate([], [x, y, z], [x, y, z])
Exit: (3) concatenate([c], [x, y, z], [c, x, y, z])
Exit: (2) concatenate([b, c], [x, y, z], [b, c, x, y, z])
Exit: (1) concatenate([a, b, c], [x, y, z], [a, b, c, x, y, z])
Output = [a, b, c].
```

```
[trace] ?- concatenate(O1,O2,[a,b,c]).
```

```
...
```

```
O1 = [],
```

```
O2 = [a, b, c] ;
```

```
...
```

```
O1 = [a],
```

```
O2 = [b, c] ;
```

```
...
```

```
O1 = [a, b],
```

```
O2 = [c] ;
```

```
...
```

```
O1 = [a, b, c],
```

```
O2 = [] ;
```

```
...
```

```
false.
```

```
% Program to reverse a list and return the result in a second list.
reverse([], []).
reverse([Head|Tail], Z) :- reverse(Tail, Y), concatenate(Y, [Head], Z).
```

AND and OR

In Prolog, the concept of "and" is implemented using a comma. We have seen this a few times so far. For example, the predicate:

```
sibling(X, Y) :- parent(Z, X), parent(Z, Y), X \= Y.
```

shows that for X and Y to be siblings, the parent of X AND the parent of Y must be the same Z AND X can't be equal to Y. So, every time a comma is used, we view this as an AND condition.

The concept of "or" is implemented in one of two ways. Either using the semicolon. We have seen this when Prolog gives us an answer to our query. We type semicolon to ask about other potential answers. For example, given the rules:

```
parent(bob, ann).
parent(bob, pat).
```

Asking Prolog the following query and hitting semicolon reveals two answers:

```
?- parent(bob, X).
X = ann ;
X = pat.
```

Which means that X can be equal to ann OR X can be equal to pat. Either one would make the statement true. We can also use this in a rule:

```
ancestor(X, Z) :- parent(X, Z) ;
                  parent(X, Y), ancestor(Y, Z).
```

This means that X is an ancestor of Z if X is the parent of Z OR X is the parent of Y who is the ancestor of Z.

We can also represent OR just by putting the same rule twice with different conditions:

```
ancestor(X, Z) :- parent(X, Z).
ancestor(X, Z) :- parent(X, Y), ancestor(Y, Z).
```

This accomplishes the same thing and still implements an OR condition.

Operators

Operator	Constraint Meaning	Can Perform Unification?
=	basic equality (with unification). No arithmetic evaluation. Attempt to unify terms together, fails if it can't	Yes
==	basic equality. No arithmetic evaluation. Are terms already unified and equivalent? Fail if not and don't try to unify them if they already aren't.	No
\=	basic inequality. No arithmetic evaluation. Are terms already unified and not equivalent? Fail if either term isn't unified already.	No
is	one sided arithmetic equality. Evaluate the right hand side as an arithmetic expression and unify it with the term on the left hand side (if it is a variable). Fail if can't.	Yes
:=	arithmetic equality. Evaluate both sides as arithmetic expressions. Result in true if the evaluated values are the same. Fail if any variables aren't unified already.	No
=\=	arithmetic inequality. Evaluate both sides as arithmetic expressions. Result in true if the evaluated values are NOT the same. Fail if any variables aren't unified already.	No
<	Performs evaluation on both sides if needed, first term should be less than second term.	No
=<	Performs evaluation on both sides if needed, first term should be less than or equal to second term.	No
>	Performs evaluation on both sides if needed, first term should be greater than second term.	No
>=	Performs evaluation on both sides if needed, first term should be greater than or equal to second term.	No

Examples for "is", "=", "==":

```
% First, since "is" is used, we perform arithmetic and get the answer 2.
% So, X is unified with the atom '2'. For == the terms must already be
% unified, which they are and X was previously bound to '2' so this succeeds.
?- X is 2/2*2, X == 2.
X = 2.
```

```
% Remember that for == the terms must already have be unified and then be
% equivalent. Hence, since X hasn't been bound to an atom yet, X == 2 fails.
?- X == 2, X is 2/2*2.
false.
```

```
% For = the terms will try to be unified together.
% Hence X will be unified to 2 and then X is 2/2*2 will attempt to match.
% Since "is" evaluates arithmetic, after evaluation we get '2'.
% So X = 2 is a possible answer.
?- X = 2, X is 2/2*2.
X = 2.
```

```
% Similar to before, X will first be unified with 2.
% But then, X is attempted to be unified with 2/2*2.
% Since "is" was not used, no arithmetic is performed, and we are
% attempting to unify X with the atom '2/2*2' which is a different atom to
% just '2' by itself. X can't be both values, so unification fails.
% Even though atoms and strings are two different things, this example may
% make more sense if you imagine "2" and "2/2*2" were strings.
% Clearly "2" and "2/2*2" are not the same strings, so this fails.
?- X = 2, X = 2/2*2.
false.
```

Examples for "\=", "<=", ">=":

```
% This fails even though there are clearly values for X that are less than
% or equal to 10, greater than or equal to 0 and not 5.
% Remember that unification isn't triggered by these operators, so it
% won't go through all possibilities to find values for X.
```

```
?- X =< 10, X >= 0, X \== 5.
```

```
ERROR: Arguments are not sufficiently instantiated
```

```
ERROR: In:
```

```
ERROR:    [11] _92=<10
```

```
ERROR:    [10] ' <meta-call>' ('<garbage_collected>') <foreign>
```

```
ERROR:    [9] toplevel_call('<garbage_collected>') at c:/program
files/swipl/boot/toplevel.pl:1173
```

```
% However, if X is already unified, we can check if the atom it is bound
% to meets the conditions. Here it does.
```

```
?- X = 2, X =< 10, X >= 0, X \= 5.
```

```
X = 2.
```

```
% X is bound to 5 but this fails the X \== 5 constraint.
```

```
?- X = 5, X =< 10, X >= 0, X \= 5.
```

```
false.
```

```
% X is bound to 11 which fails the X =< 10 constraint.
```

```
?- X = 11, X =< 10, X >= 0, X \= 5.
```

```
false.
```

```
% X is bound to -1 which fails the X >= 0 constraint.
```

```
?- X = -1, X =< 10, X >= 0, X \= 5.
```

```
false.
```

```
% If we want to enumerate the possibilities for X, we can use a predicate
% instead of a built-in operator. The predicate "between" will perform
% unification and try all possibilities
```

```
?- between(0, 10, X), X \= 5.
```

```
X = 0 ;
```

```
X = 1 ;
```

```
X = 2 ;
```

```
X = 3 ;
```

```
X = 4 ;
```

```
X = 6 ;
```

```
X = 7 ;
```

```
X = 8 ;
```

```
X = 9 ;
```

```
X = 10.
```

The Cut Operator

Sometimes we want to "force" the direction of a computation and prevent the underlying Prolog engine from backtracking to try different solution paths. The cut operator, written as "!", provides a way of preventing Prolog from backtracking. This operator is useful when you want to limit Prolog to a single answer. The "occurs" procedure, given below, which counts how many times an item occurs in a list is a good example of a Prolog rule that employs cut.

```
% Program that accepts an element and a list. The program returns a count
% of the number of times that the element appears in the list.

% Note: It is necessary to use a cut operator in this definition of occurs
% to prevent backtracking (and therefore the return of multiple answers).
% This can easily be understood by noting that anything that matches the
% head (conclusion) of the second rule will automatically also satisfy the
% head of the third rule, since it is more general. The cut (!) in the
% second rule will prevent Prolog from backing up and trying the
% alternative offered by the third rule.

occurs(X, [], 0).
occurs(X, [X|Tail], Y)      :- !, occurs(X, Tail, Num), Y is Num + 1.
occurs(X, [Head|Tail], Y) :- occurs(X, Tail, Y).
```

Let's see this in action. Asking Prolog how many times 'a' appears in [a,b,a,a], we get:

```
?- occurs(a, [a,b,a,a], Num).
Num = 3 ;
false.
```

Prolog correctly tells us it appears 3 times. We then type semi colon to see if there are other bindings that would make this true and Prolog can't find any. It can't find any because the cut operator is not allowing it to explore other possibilities.

Now let's try that same query but this time we remove the cut operator from our occurs rule and reload our database file.

```
?- occurs(a, [a,b,a,a], Num).
Num = 3 ;
Num = 2 ;
Num = 2 ;
Num = 1 ;
Num = 2 ;
Num = 1 ;
Num = 1 ;
Num = 1 ;
Num = 0 ;
false.
```

Backtracking is a powerful part of Prolog, but in this case, it just causes confusion as to what the real answer is. So, the cut operator is there to help us turn that feature off for any solution paths that cross the cut operator.

Don't forget that we can ask a question in ANY direction. This means we can also do the following:

```
?- occurs(a, L, 3).  
L = [a, a, a] .
```

```
?- occurs(X, [a,b,a,a], 3).  
X = a .
```

However, this doesn't seem to work as well with other queries, for example:

```
?- occurs(X, [a,b,a,a], 1).  
false.
```

As stated before, Prolog attempts to be as declarative as possible. This should mean that certain implementation choices and details shouldn't matter as much. Unfortunately, in reality they do. In the query above, Prolog first unifies X with 'a' and tries to verify this as a solution. This doesn't work since there are too many a's. However, the cut operator prevents us from trying other possibilities and so we conclude failure.

Simply removing the cut is the obvious choice to fix this:

```
occurs(X, [], 0).  
occurs(X, [X|Tail], Y) :- occurs(X, Tail, Num), Y is Num + 1.  
occurs(X, [Head|Tail], Y) :- occurs(X, Tail, Y).
```

But then what happens is that X becomes bound to everything that has AT LEAST 1 of the desired atom in the list. In other words, X is unified with 'a' as a solution, then upon backtracking is unified with 'b' as another solution (in fact, X = a comes up several times if you cycle through the answers with semicolon).

To fix this, we need to enforce that X can't be equal to the Head in the third rule:

```
occurs(X, [], 0).  
occurs(X, [X|Tail], Y) :- occurs(X, Tail, Num), Y is Num + 1.  
occurs(X, [Head|Tail], Y) :- X \= Head, occurs(X, Tail, Y).
```

However, recall that \= won't cause unification to occur. So unless X and Head were already unified before this goal is reached, it will fail. To fix this, we must put the goal at the end (at the point where X and Head would have been unified already).

```
occurs(X, [], 0).  
occurs(X, [X|Tail], Y) :- occurs(X, Tail, Num), Y is Num + 1.  
occurs(X, [Head|Tail], Y) :- occurs(X, Tail, Y), X \= Head.
```

So, we got rid of the cut operator and ensured that X and Head can't be bound to the same atom if using the third rule. Now we get sensible answers to our question, regardless of the order asked:

```
?- occurs(a, [a,b,c,c,b,c], 1).  
true ;  
false.
```

```
?- occurs(c, [a,b,c,c,b,c], 3).
true ;
false.
```

```
?- occurs(c, [a,b,c,c,b,c], 4).
false.
```

```
?- occurs(b, [a,b,c,c,b,c], N).
N = 2 ;
false.
```

```
?- occurs(X, [a,b,c,c,b,c], 1).
X = a ;
false.
```

```
?- occurs(X, [a,b,c,c,b,c], 2).
X = b ;
false.
```

```
?- occurs(X, [a,b,c,c,b,c], 3).
X = c ;
false.
```

```
?- occurs(X, [a,b,c,c,b,c], 4).
false
```

Notice that I am hitting semicolon and not getting any other answers. This is good as it means I'm filtering out incorrect answers. I can even ask what list I would have to have to satisfy 4 d's:

```
?- occurs(d, L, 4).
L = [d, d, d, d] .
```

For this one I hit enter instead of semicolon. Semicolon makes it hang while generating an infinite number of possibilities. So, while not a perfect definition of "occurs", it works for most cases.

Singleton Variables

When compiling the occurs rule, you probably got the following warning:

```
% in main.pl

occurs(X, [], 0).
occurs(X, [X|Tail], Y)      :- occurs(X, Tail, Num), Y is Num + 1.
occurs(X, [Head|Tail], Y) :- occurs(X, Tail, Y), X \= Head.

% in SWI terminal

?- ['main'].
Warning: main.pl:1:
Warning: Singleton variables: [X]
true.
```

This just means that there is a variable in our rule that we are not actually using. As we can see, rule 1 doesn't really use X. We can get rid of these warnings by replacing the unused variables with underscores _

```
% in main.pl
occurs(_, [], 0).
occurs(X, [X|Tail], Y) :- occurs(X, Tail, Num), Y is Num + 1.
occurs(X, [Head|Tail], Y) :- occurs(X, Tail, Y), X \= Head.

% in SWI terminal
?- ['main'].
true.
```

Terminology

Facts and rules collectively are called clauses. If a clause only has a "head" and no body, it is a fact. If a clause has a head and body, it is a rule. A body is composed of goals. This is the same for a query, which is also composed of goals. Multiple clauses with the same name form a predicate. For example:

```
female(pam).
female(liz).
female(ann).
female(pat).
```

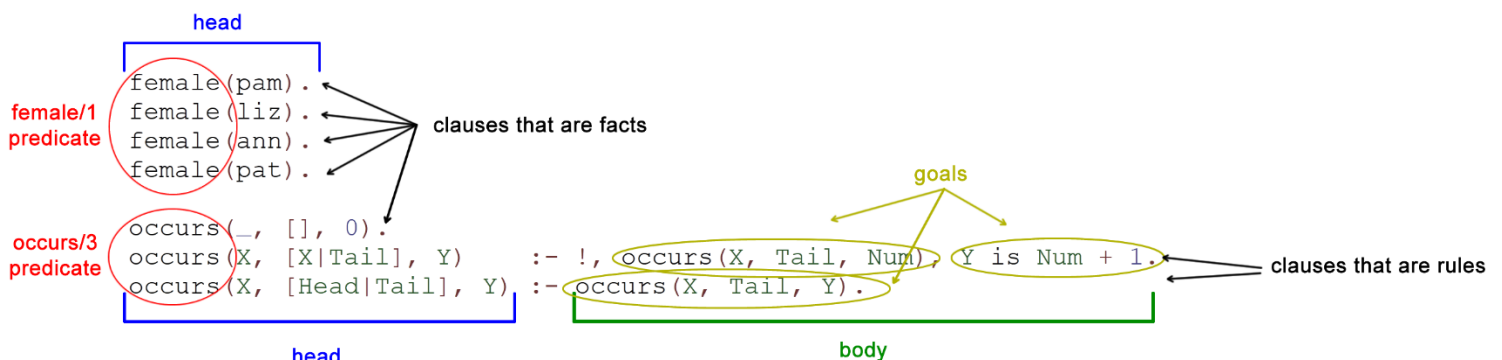
female is a predicate with an "arity" of 1 (meaning it has 1 argument). We would write this as `female/1`. "female(pam)" is the head of a clause. This clause doesn't have a body, so it is a fact. There are 4 total clauses to the female predicate.

```
occurs(_, [], 0).
occurs(X, [X|Tail], Y) :- !, occurs(X, Tail, Num), Y is Num + 1.
occurs(X, [Head|Tail], Y) :- occurs(X, Tail, Y).
```

occurs is a predicate with an arity of 3 (i.e. `occurs/3`). The first occurs clause is a fact. The other two clauses are rules since they have both a head and a body. Within the body of the second occurs clause, there are three goals (e.g. "Y is Num + 1" is one of the goals).

The collection of all predicates (and the clauses they are composed of) form a Prolog database, which some people call a "program".

So, in summary:



Helpful Predicates²

Information

Predicate	Meaning	Example
<code>listing/1</code>	pull up defined clauses of a predicate. include its arity to get only clauses with that arity, otherwise leave that part off to get all clauses.	<code>listing(member/2).</code> <code>listing(append/3).</code> <code>listing(listing).</code>

Data Type Checking and Conversion

Predicate	Meaning	Example
<code>atom/1</code>	true if argument is an atom (or can be unified to an atom)	<code>atom(42).</code> % false <code>atom('42').</code> % true <code>atom(something).</code> % true <code>atom("something").</code> % false
<code>string/1</code>	true if argument is a string (or can be unified to a string)	<code>string(42).</code> % false <code>string('42').</code> % false <code>string(something).</code> % false <code>string("something").</code> % true
<code>number/1</code>	true if argument is a number (or can be unified to a number)	<code>number(42).</code> % true <code>number(42.5).</code> % true <code>number('42').</code> % false
<code>integer/1</code>	true if argument is an integer (or can be unified to an integer)	<code>integer(42).</code> % true <code>integer(42.5).</code> % false <code>integer('42').</code> % false
<code>atom_string/2</code>	converts an atom to a string or vice versa	<code>atom_string(abc, S).</code> % S = "abc" <code>atom_string(A, "abc").</code> % A = abc
<code>atom_number/2</code>	converts an atom to a number or vice versa	<code>atom_number('5', N).</code> % N = 5 <code>atom_number(A, 5).</code> % A = '5'
<code>number_string/2</code>	converts a number to a string or vice versa	<code>number_string(123, S).</code> % S = "123" <code>number_string(N, "123").</code> % N = 123

String Processing

Predicate	Meaning	Example
<code>re_match/2</code>	true if given regex matches text	<code>re_match("[a-z]+"/i, "myVariable").</code>
<code>re_split/4</code>	uses regex to split a string into parts	<code>re_split("\\s*\\W\\s*", Line, RawTokens, []).</code>
<code>normalize_space/2</code>	trims trailing and leading white space from string and gives it back as an atom.	<code>normalize_space(atom(N), " x ").</code> % N = 'x'

² <https://www.swi-prolog.org/pldoc/man?section=builtin>

Examples

% EXAMPLE 1: Towers of Hanoi

```
hanoi(_, _, _, NumDisks) :- NumDisks < 1,
                           write("must have at least 1 disk").

hanoi(Src, _, Dst, NumDisks) :- NumDisks = 1,
                                format('~w -> ~w', [Src, Dst]),
                                nl.

hanoi(Src, Spr, Dst, NumDisks) :- NumDisks > 1,
                                OneLessDisk is NumDisks - 1,
                                hanoi(Src, Dst, Spr, OneLessDisk),
                                hanoi(Src, Spr, Dst, 1),
                                hanoi(Spr, Src, Dst, OneLessDisk).
```

% EXAMPLE 2a: Finding the minimum value in a list

```
% unifies CurMin with head of list to start the process
list_min1([H|T], Min) :- list_min1(T, H, Min).

% unifies CurMin to actual Min when list is empty
list_min1([], CurMin, CurMin).

list_min1([H|T], CurMin, Min) :-
    % uses built-in min to set NewMin to the smaller of CurMin and head of list
    NewMin is min(H, CurMin),

    % recursively call with NewMin and rest of list
    list_min1(T, NewMin, Min).
```

% EXAMPLE 2b: (Another way to) find the minimum value in a list

```
% when list has one item left, unify it with Min
list_min2([Min], Min).

% if first item in list is smaller than second item,
% recursively call predicate and discard second item
list_min2([H,K|T], M) :- H =< K, list_min2([H|T], M).

% if first item in list is larger than second item,
% recursively call predicate and discard first item
list_min2([H,K|T], M) :- H > K, list_min2([K|T], M).
```

% EXAMPLE 3: Getting length of a list

% when list is empty, unify OldLen with 0
list_len([], 0).

% when list has at least one item

list_len([_|T], Len) :-
 % recursively call with rest of the list
 list_len(T, OldLen),

% when we get back from recursion, add 1 to OldLen to get current length
% when recursion fully unwinds and we go back to initial query, Len will
% have the full length of the list
Len is OldLen + 1.

% EXAMPLE 4: Calculating the sum of all values in a list

list_sum([], 0).
list_sum([H|T], Sum) :- list_sum(T, Sum1),
 Sum is Sum1 + H.

% EXAMPLE 5: Calculating the average of all values in a list

list_avg(L, Avg) :- list_sum(L, Sum),
 list_len(L, Len),
 Avg is Sum / Len.

% EXAMPLE 6: Testing custom list_len predicate against built-in length predicate

test(Start, Stop) :- Start > Stop,
 !,
 fail.

test(Start, Stop) :- list_len(L, Start),
 test(L, Start, Stop, Start).

test(_, _, Stop, Cur) :- Cur > Stop,
 !,
 write("done").

test(List, Start, Stop, Cur) :-
 Cur =< Stop,
 length(List, L),
 list_len(List, L),
 format("Test with ~w items (~w) passed ~n", [Cur, List]),
 NextCur is Cur + 1,
 append(List, [a], NewList),
 test(NewList, Start, Stop, NextCur).

% EXAMPLE 7: parsing print statement with numeric literals

% matches print token as string

```
print("print").
```

% matches print token as atom

```
print('print').
```

% matches numbers

```
num_val(Num, Val) :-
```

```
    % if number is an atom (e.g. '55'), converts to numeric value (e.g. 55)
    atom(Num), atom_number(Num, Val) ;
```

```
    % OR if number is a string (e.g. "55"), converts to numeric value (e.g. 55)
    string(Num), number_string(Val, Num) ;
```

```
    % OR if number is an integer (e.g. 55), copies numeric value
    integer(Num), Val is Num.
```

% checks to ensure that Print variable is the print token and

% that Num variable is a number, then output the numeric value

```
print_num(Print, Num) :- print(Print),
                           num_val(Num, Val),
                           format("Output: ~w ~n", [Val]).
```

% queries

```
?- print_num(print, 56).
```

Output: 56

true.

```
?- print_num('print', '983').
```

Output: 983

true .

```
?- print_num(print, "0").
```

Output: 0

true .

```
?- print_num(print, abc).
```

false.

```
?- print_num(prints, 2).
```

false.

Refs

- <https://www.metalevel.at/prolog>
- <https://www.youtube.com/@PlayingWithProlog/videos>
- <https://www.youtube.com/@ThePowerOfProlog/videos>
- [https://www.reddit.com/r/prolog/comments/ba4hn3/beginner to intermediate project ideas/](https://www.reddit.com/r/prolog/comments/ba4hn3/beginner_to_intermediate_project_ideas/)
- <https://www.youtube.com/watch?v=SykxWpFwMGs>
- <https://www.amzi.com/AdventureInProlog/a1start.php>